UNIT - III

Planning a software Project: Effort Estimation, Project Schedule and Staffing, Quality Planning, Risk Management. **Estimation for Software Projects:** Decomposition Techniques - Software Sizing, Problem-Based Estimation, an Example of LOC-Based Estimation, an Example of FP-Based Estimation, COCOMO Model

Effort estimation:

Effort estimation is a crucial aspect of software engineering as it helps in planning, budgeting, and resource allocation for projects. Accurate effort estimation ensures that projects are delivered on time and within budget. Here are some common techniques and approaches used in software engineering for effort estimation:

1. Expert Judgment

- **Description:** Leveraging the experience and intuition of seasoned experts in the field to estimate effort.
- **Advantages:** Quick and often accurate if experts have relevant experience.
- **Disadvantages:** Subjective and can be biased; relies heavily on the availability of experienced personnel.

2. Analogous Estimation

- **Description:** Using historical data from similar past projects to estimate the effort for the current project.
- **Advantages:** Based on actual data; useful when there are similarities between projects.
- **Disadvantages:** Requires accurate historical data; differences between projects can lead to inaccuracies.

3. Parametric Estimation

- **Description:** Using mathematical models to predict effort based on project parameters (e.g., lines of code, function points).
- **Advantages:** Can be highly accurate if the model is well-calibrated; scalable.
- **Disadvantages:** Requires accurate data and parameters; models need to be validated and maintained.

4. Bottom-Up Estimation

- **Description:** Breaking down the project into smaller tasks and estimating the effort for each task individually before summing them up.
- **Advantages:** Detailed and can provide high accuracy; considers all aspects of the project.
- **Disadvantages:** Time-consuming and requires detailed project decomposition.

5. Top-Down Estimation

- **Description:** Estimating the overall effort for the entire project and then breaking it down into smaller components.
- **Advantages:** Quick and useful for early-stage estimates; provides a high-level view.
- **Disadvantages:** Can miss details and may lead to inaccuracies if the overall estimate is off.

6. Function Point Analysis (FPA)

- **Description:** Estimating effort based on the functionality provided to the user, measured in function points.
- **Advantages:** Independent of programming language; focuses on user requirements.
- **Disadvantages:** Requires experience in function point counting; can be complex to apply.

7. Use Case Points (UCP)

- **Description:** Estimating effort based on the number and complexity of use cases in the system.
- **Advantages:** Tied directly to user requirements; relatively straightforward.
- **Disadvantages:** Can be subjective; requires accurate assessment of use case complexity.

8. Wideband Delphi Technique

- **Description:** A consensus-based estimation method where experts provide estimates anonymously, discuss differences, and iterate until a consensus is reached.
- **Advantages:** Reduces bias and leverages multiple viewpoints; can improve accuracy.
- **Disadvantages:** Time-consuming; relies on the availability of experts.

9. COCOMO (Constructive Cost Model)

- **Description:** A parametric model that estimates effort based on the size of the software and a set of cost drivers.
- **Advantages:** Based on extensive empirical data; provides detailed and structured estimates.
- **Disadvantages:** Complex to implement; requires calibration to the organization's environment.

10. Planning Poker

- **Description:** A consensus-based agile estimation technique where team members estimate effort by playing cards with values representing effort.
- **Advantages:** Promotes team involvement and discussion; quick and engaging.
- **Disadvantages:** Can be less accurate for large projects; relies on team consensus.

Steps to Improve Effort Estimation Accuracy

1. **Gather Historical Data:**
    - Collect and analyze data from past projects to inform future estimates.

2. **Involve the Right People:**
    - Engage team members with relevant expertise and experience in the estimation process.
3. **Use Multiple Methods:**
    - Combine different estimation techniques to cross-verify and improve accuracy.
4. **Regularly Update Estimates:**
    - Revise estimates as the project progresses and more information becomes available.
5. **Consider Risk and Uncertainty:**
    - Include contingencies and buffer times to account for potential risks and uncertainties.
6. **Track and Analyze Performance:**
    - Monitor actual effort against estimates and analyze deviations to refine estimation methods.

Project Schedule and Staffing:

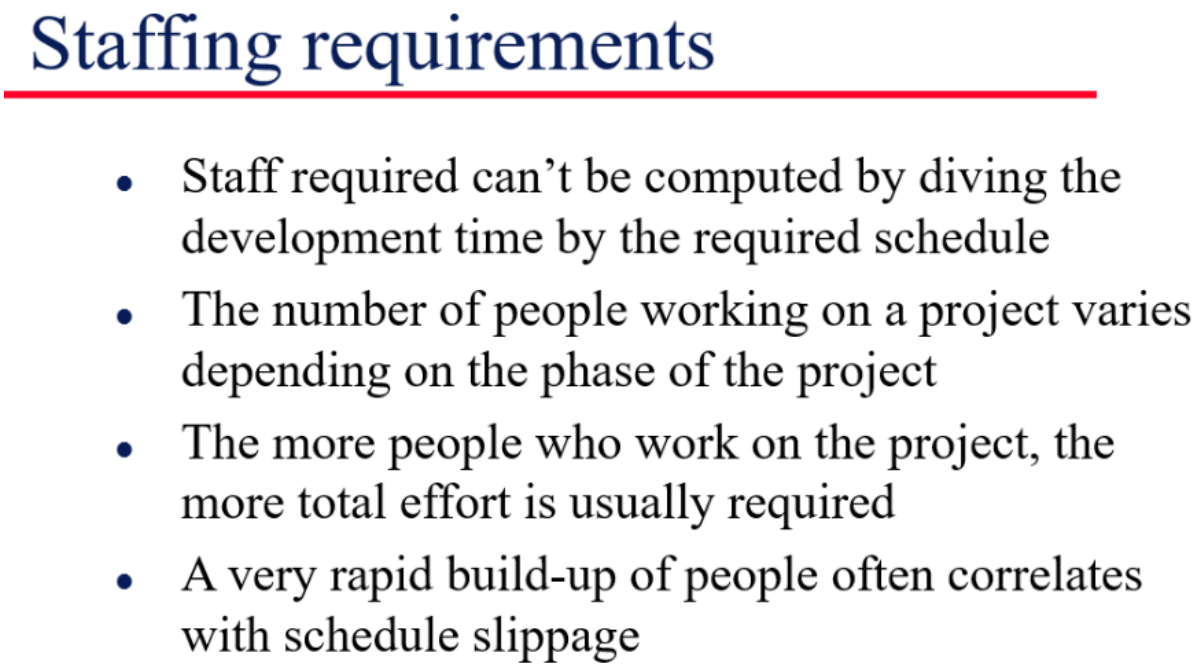
In software engineering, a well-organized project schedule and appropriate staffing are crucial for the successful completion of a project. Below are the steps and considerations for developing an effective project schedule and staffing plan:

Project Schedule

1. **Define Project Scope:**
    - Clearly outline the project objectives, deliverables, and constraints.
    - Identify the key milestones and phases of the project.
2. **Break Down Tasks:**
    - Use a Work Breakdown Structure (WBS) to divide the project into manageable tasks.
    - Ensure that each task is specific, measurable, achievable, relevant, and time-bound (SMART).
3. **Sequence Tasks:**
    - Determine the dependencies between tasks.
    - Create a task sequence that ensures logical progression.
4. **Estimate Duration:**
    - Estimate the time required to complete each task.
    - Use techniques like expert judgment, analogous estimation, parametric estimation, and three-point estimation.
5. **Allocate Resources:**
    - Assign resources (personnel, equipment, software) to each task.
    - Consider the availability and skill set of team members.
6. **Create a Timeline:**
    - Use Gantt charts or network diagrams to visualize the schedule.
    - Set start and end dates for each task, ensuring milestones are met.
7. **Set Milestones:**
    - Define key milestones to track progress.
    - Ensure milestones are achievable and align with project goals.

8. **Risk Management:**
    - Identify potential risks and develop mitigation plans.
    - Include buffer times for high-risk tasks.
9. **Review and Adjust:**
    - Regularly review the schedule and adjust as necessary.
    - Use project management tools like Microsoft Project, JIRA, or Trello for tracking and updates.

Staffing Plan



1. **Identify Roles and Responsibilities:**
    - Define the roles required for the project (e.g., project manager, developers, testers, designers).
    - Clearly outline responsibilities for each role.
2. **Assess Skills and Experience:**
    - Evaluate the skills and experience of available team members.
    - Match tasks to team members' strengths and areas of expertise.
3. **Resource Allocation:**
    - Assign tasks to team members based on their skills and availability.
    - Ensure balanced workload distribution to avoid burnout.
4. **Recruitment:**
    - Identify any skill gaps and plan for hiring or contracting additional resources.
    - Consider temporary contractors or consultants if specialized skills are needed for short periods.
5. **Training and Development:**
    - Provide necessary training to team members to bridge any skill gaps.
    - Encourage continuous learning and professional development.
6. **Communication Plan:**
    - Establish clear communication channels and protocols.

    - Regularly update the team on project progress and changes.
7. **Performance Monitoring:**
    - Track individual and team performance against set goals.
    - Provide regular feedback and support to ensure high performance.
8. **Adjust Staffing as Needed:**
    - Be flexible and adjust the staffing plan as project needs evolve.
    - Consider adding or reallocating resources based on project demands.

Tools and Techniques

- **Project Management Software:**
    - Tools like Microsoft Project, JIRA, Asana, Trello, and Monday.com help in planning, scheduling, and tracking tasks and resources.
- **Communication Tools:**
    - Use Slack, Microsoft Teams, or Zoom for team communication and collaboration.
- **Documentation Tools:**
    - Maintain documentation with Confluence, Google Docs, or SharePoint.

Effective project scheduling and staffing are essential for delivering software projects on time and within budget. By following these guidelines, you can enhance the organization, communication, and execution of your software engineering projects.

Quality Planning:

Quality planning is a fundamental aspect of software engineering that ensures the final product meets the required standards and satisfies user needs. It involves setting quality goals, defining quality metrics, and outlining the processes to achieve and maintain quality throughout the software development lifecycle. Here's an overview of key components and steps in quality planning:

Key Components of Quality Planning

1. **Quality Objectives:**
    - Define clear and measurable quality goals that the project aims to achieve.
    - Ensure objectives align with customer requirements and organizational standards.
2. **Quality Standards:**
    - Identify and adhere to relevant industry standards (e.g., ISO, IEEE).
    - Incorporate organizational quality policies and procedures.
3. **Quality Metrics:**
    - Establish metrics to measure product quality (e.g., defect density, code coverage, mean time to failure).
    - Use both process metrics (e.g., review efficiency, test case execution rate) and product metrics.
4. **Quality Assurance (QA) Activities:**
    - Define processes for ensuring quality during development (e.g., code reviews, testing).
    - Include continuous integration and continuous deployment (CI/CD) practices.
5. **Quality Control (QC) Activities:**

    - Plan for activities to verify that the product meets quality standards (e.g., inspections, walkthroughs, testing).
    - Ensure QC activities are performed at different stages of the development lifecycle.
6. **Roles and Responsibilities:**
    - Assign specific quality-related roles (e.g., QA lead, testers).
    - Define the responsibilities of each team member in maintaining quality.
7. **Tools and Techniques:**
    - Select appropriate tools for quality management (e.g., test management tools, static analysis tools).
    - Use techniques such as root cause analysis, Six Sigma, and Total Quality Management (TQM).

Steps in Quality Planning

1. **Requirement Analysis:**
    - Gather and analyze user requirements to understand quality expectations.
    - Ensure requirements are clear, complete, and testable.
2. **Define Quality Criteria:**
    - Establish criteria for product acceptance and performance.
    - Include functional and non-functional requirements (e.g., usability, performance, security).
3. **Develop Quality Plan:**
    - Document the quality objectives, standards, metrics, and processes in a comprehensive quality plan.
    - Ensure the plan is reviewed and approved by all stakeholders.
4. **Design Quality Processes:**
    - Design processes for software development that incorporate quality practices.
    - Include processes for requirements management, design, coding, testing, and maintenance.
5. **Implement Quality Processes:**
    - Ensure quality processes are integrated into the development workflow.
    - Provide training and resources to the team to follow quality practices.
6. **Monitor and Measure Quality:**
    - Continuously monitor quality metrics and processes.
    - Use dashboards and reports to track progress and identify areas for improvement.
7. **Conduct Quality Audits and Reviews:**
    - Regularly conduct audits and reviews to assess compliance with quality standards.
    - Use findings to improve processes and address any gaps.
8. **Manage Changes:**
    - Implement a change management process to handle changes in requirements and scope.
    - Ensure changes are evaluated for their impact on quality and documented appropriately.
9. **Continuous Improvement:**
    - Foster a culture of continuous improvement by regularly evaluating and refining quality processes.

- Encourage team feedback and learning from past projects to enhance future quality planning.

Quality Planning Tools

1. **Test Management Tools:**
   - Tools like TestRail, Zephyr, and QTest help manage test cases, plans, and execution.
2. **Static Analysis Tools:**
   - Tools like SonarQube and Coverity analyze code for potential defects and code quality issues.
3. **Continuous Integration/Continuous Deployment (CI/CD) Tools:**
   - Tools like Jenkins, Travis CI, and CircleCI automate build, test, and deployment processes.
4. **Requirement Management Tools:**
   - Tools like JIRA, Confluence, and IBM DOORS help manage and track requirements.
5. **Quality Management Systems (QMS):**
   - Systems like ISO 9001 provide frameworks for establishing and maintaining quality management processes.

What is Risk?

"Tomorrow problems are today's risk." Hence, a clear definition of a "risk" is a problem that could cause some loss or threaten the progress of the project, but which has not happened yet.

These potential issues might harm cost, schedule or technical success of the project and the quality of our software device, or project team morale.

Risk Management is the system of identifying addressing and eliminating these problems before they can damage the project.

Risk Management

A software project can be concerned with a large variety of risks. In order to be adept to systematically identify the significant risks which might affect a software project, it is essential to classify risks into different classes. The project manager can then check which risks from each class are relevant to the project.

There are three main classifications of risks which can affect a software project:

1. Project risks
2. Technical risks
3. Business risks

1. Project risks: Project risks concern differ forms of budgetary, schedule, personnel, resource, and customer-related problems. A vital project risk is schedule slippage. Since the software is intangible, it is very tough to monitor and control a software project. It is very tough to control something which cannot be identified. For any manufacturing program, such as the manufacturing of cars, the plan executive can recognize the product taking shape.

2. Technical risks: Technical risks concern potential method, implementation, interfacing, testing, and maintenance issue. It also consists of an ambiguous specification, incomplete specification, changing specification, technical uncertainty, and technical obsolescence. Most technical risks appear due to the development team's insufficient knowledge about the project.

3. Business risks: This type of risks contain risks of building an excellent product that no one need, losing budgetary or personnel commitments, etc.

Other risk categories

1. **1. Known risks:** Those risks that can be uncovered after careful assessment of the project program, the business and technical environment in which the plan is being developed, and more reliable data sources (e.g., unrealistic delivery date)
2. **2. Predictable risks:** Those risks that are hypothesized from previous project experience (e.g., past turnover)
3. **3. Unpredictable risks:** Those risks that can and do occur, but are extremely tough to identify in advance.

Principle of Risk Management

1. **Global Perspective:** In this, we review the bigger system description, design, and implementation. We look at the chance and the impact the risk is going to have.
2. **Take a forward-looking view:** Consider the threat which may appear in the future and create future plans for directing the next events.
3. **Open Communication:** This is to allow the free flow of communications between the client and the team members so that they have certainty about the risks.
4. **Integrated management:** In this method risk management is made an integral part of project management.
5. **Continuous process:** In this phase, the risks are tracked continuously throughout the risk management paradigm.

Risk Management Activities



Risk Assessment

The objective of risk assessment is to division the risks in the condition of their loss, causing potential. For risk assessment, first, every risk should be rated in two methods:

- The possibility of a risk coming true (denoted as r).
- The consequence of the issues relates to that risk (denoted as s).

Based on these two methods, the priority of each risk can be estimated:

p = r * s

Where p is the priority with which the risk must be controlled, r is the probability of the risk becoming true, and s is the severity of loss caused due to the risk becoming true. If all identified risks are set up, then the most likely and damaging risks can be controlled first, and more comprehensive risk abatement methods can be designed for these risks.

1. Risk Identification: The project organizer needs to anticipate the risk in the project as early as possible so that the impact of risk can be reduced by making effective risk management planning.

A project can be of use by a large variety of risk. To identify the significant risk, this might affect a project. It is necessary to categories into the different risk of classes.

There are different types of risks which can affect a software project:

1. **Technology risks:** Risks that assume from the software or hardware technologies that are used to develop the system.
2. **People risks:** Risks that are connected with the person in the development team.
3. **Organizational risks:** Risks that assume from the organizational environment where the software is being developed.
4. **Tools risks:** Risks that assume from the software tools and other support software used to create the system.
5. **Requirement risks:** Risks that assume from the changes to the customer requirement and the process of managing the requirements change.
6. **Estimation risks:** Risks that assume from the management estimates of the resources required to build the system

2. Risk Analysis: During the risk analysis process, you have to consider every identified risk and make a perception of the probability and seriousness of that risk.

There is no simple way to do this. You have to rely on your perception and experience of previous projects and the problems that arise in them.

It is not possible to make an exact, the numerical estimate of the probability and seriousness of each risk. Instead, you should authorize the risk to one of several bands:

1. The probability of the risk might be determined as very low (0-10%), low (10-25%), moderate (25-50%), high (50-75%) or very high (+75%).
2. The effect of the risk might be determined as catastrophic (threaten the survival of the plan), serious (would cause significant delays), tolerable (delays are within allowed contingency), or insignificant.

Risk Control

It is the process of managing risks to achieve desired outcomes. After all, the identified risks of a plan are determined; the project must be made to include the most harmful and the most likely risks. Different risks need different containment methods. In fact, most risks need ingenuity on the part of the project manager in tackling the risk.

There are three main methods to plan for risk management:

1. **Avoid the risk:** This may take several ways such as discussing with the client to change the requirements to decrease the scope of the work, giving incentives to the engineers to avoid the risk of human resources turnover, etc.
2. **Transfer the risk:** This method involves getting the risky element developed by a third party, buying insurance cover, etc.
3. **Risk reduction:** This means planning method to include the loss due to risk. For instance, if there is a risk that some key personnel might leave, new recruitment can be planned.

Risk Leverage: To choose between the various methods of handling risk, the project plan must consider the amount of controlling the risk and the corresponding reduction of risk. For this, the risk leverage of the various risks can be estimated.

Risk leverage is the variation in risk exposure divided by the amount of reducing the risk.

Risk leverage = (risk exposure before reduction - risk exposure after reduction) / (cost of reduction)

1. Risk planning: The risk planning method considers each of the key risks that have been identified and develop ways to maintain these risks.

For each of the risks, you have to think of the behavior that you may take to minimize the disruption to the plan if the issue identified in the risk occurs.

You also should think about data that you might need to collect while monitoring the plan so that issues can be anticipated.

Again, there is no easy process that can be followed for contingency planning. It rely on the judgment and experience of the project manager.

2. Risk Monitoring: Risk monitoring is the method king that your assumption about the product, process, and business risks has not changed.

Estimation for software projects:

Estimation for software projects is a critical process that involves predicting the amount of effort, time, and resources required to complete a project. Accurate estimation helps in planning, budgeting, and managing project expectations. Here are some common techniques and methodologies used for estimating software projects:

Common Estimation Techniques

1. **Expert Judgment:**
    - Involves consulting experienced team members or experts who have worked on similar projects.
    - Quick and often accurate if the experts have relevant experience.
    - **Pros:** Leverages real-world experience, useful for early-stage estimates.
    - **Cons:** Can be subjective and biased; relies heavily on the availability of experts.
2. **Analogous Estimation:**
    - Uses data from previous similar projects to estimate effort for the current project.
    - **Pros:** Based on historical data, relatively easy to implement.
    - **Cons:** Requires accurate and relevant historical data; may not account for unique project differences.
3. **Parametric Estimation:**
    - Uses mathematical models to estimate effort based on project parameters (e.g., lines of code, function points).
    - **Pros:** Scalable and can be highly accurate if the model is well-calibrated.
    - **Cons:** Requires accurate input data and validated models; may not be applicable to all projects.
4. **Bottom-Up Estimation:**
    - Breaks the project into smaller, manageable tasks and estimates the effort for each task individually before summing them up.
    - **Pros:** Detailed and can provide high accuracy; considers all project aspects.
    - **Cons:** Time-consuming and requires a detailed project breakdown.
5. **Top-Down Estimation:**

    - Estimates the overall effort for the entire project and then breaks it down into smaller components.
    - **Pros:** Quick and useful for early-stage estimates; provides a high-level view.
    - **Cons:** Can miss details and may lead to inaccuracies if the overall estimate is off.
6. **Function Point Analysis (FPA):**
    - Estimates effort based on the functionality provided to the user, measured in function points.
    - **Pros:** Independent of programming language; focuses on user requirements.
    - **Cons:** Requires experience in function point counting; can be complex to apply.
7. **Use Case Points (UCP):**
    - Estimates effort based on the number and complexity of use cases in the system.
    - **Pros:** Tied directly to user requirements; relatively straightforward.
    - **Cons:** Can be subjective; requires accurate assessment of use case complexity.
8. **Wideband Delphi Technique:**
    - A consensus-based estimation method where experts provide estimates anonymously, discuss differences, and iterate until a consensus is reached.
    - **Pros:** Reduces bias and leverages multiple viewpoints; can improve accuracy.
    - **Cons:** Time-consuming; relies on the availability of experts.
9. **COCOMO (Constructive Cost Model):**
    - A parametric model that estimates effort based on the size of the software and a set of cost drivers.
    - **Pros:** Based on extensive empirical data; provides detailed and structured estimates.
    - **Cons:** Complex to implement; requires calibration to the organization's environment.
10. **Planning Poker:**
    - A consensus-based agile estimation technique where team members estimate effort by playing cards with values representing effort.
    - **Pros:** Promotes team involvement and discussion; quick and engaging.
    - **Cons:** Can be less accurate for large projects; relies on team consensus.

Estimation Process

1. **Define Scope:**
    - Clearly outline the project objectives, deliverables, and constraints.
    - Identify the key milestones and phases of the project.
2. **Break Down Tasks:**
    - Use a Work Breakdown Structure (WBS) to divide the project into manageable tasks.
    - Ensure that each task is specific, measurable, achievable, relevant, and time-bound (SMART).
3. **Choose Estimation Techniques:**
    - Select appropriate estimation techniques based on project size, complexity, and available data.
    - Consider using a combination of methods to cross-verify estimates.
4. **Gather Data:**
    - Collect historical data from previous projects and use it as a reference.

    - Consult with experts and team members to gather their input.
5. **Estimate Effort:**
    - Apply the chosen estimation techniques to estimate the effort for each task.
    - Sum up the individual estimates to get the total project effort.
6. **Adjust for Risk and Uncertainty:**
    - Include buffer times for high-risk tasks and account for uncertainties.
    - Use risk management techniques to identify and mitigate potential risks.
7. **Validate Estimates:**
    - Review and validate the estimates with the project team and stakeholders.
    - Adjust estimates based on feedback and additional information.
8. **Document and Communicate:**
    - Document the estimation process, assumptions, and final estimates.
    - Communicate the estimates to all stakeholders and ensure they are understood and agreed upon.
9. **Monitor and Re-estimate:**
    - Continuously monitor the project progress and compare actual effort against estimates.
    - Update estimates as necessary based on changes in scope, requirements, or project conditions.

Tools for Estimation

1. **Project Management Software:**
    - Tools like Microsoft Project, JIRA, Asana, and Trello help manage and track tasks, resources, and timelines.
2. **Spreadsheet Applications:**
    - Excel or Google Sheets can be used to create and maintain estimation models and calculations.
3. **Estimation Software:**
    - Tools like Function Point Modeler, COCOMO II Suite, and Construx Estimate help with specific estimation methodologies.

Decomposition Techniques

- **Software Project Estimation** is a form of problem solving (To estimate cost, time & efforts in software project.)
- **Decomposition Technique** is divide & conquer approach of Software Project Estimation.
- By decomposition a project into major functions like software engineering related activities. cost, schedule & efforts estimation can be performed in stepwise manner.

➢**Decomposition Techniques are:**

1. Software Sizing
2. Problem based Estimation
3. Process based Estimation



1. Software Sizing

- Sizing represents the project planner's first major challenge.

➢**The accuracy of a software project estimate is predicated on a number of things:**

1. The degree to which the planner has properly estimated the size of the product to be built
2. The ability to translate the size estimate into human effort, calendar time and dollars.
3. The degree to which the project plan reflects the abilities of the software team.
4. The stability of product requirements & environment that supports software engineering effort.

➢**Approaches of Software Sizing:**

1. "Fuzzy Logic" Sizing
2. Function Point Sizing
3. Standard Component Sizing
4. Change Sizing

Approaches of Software Sizing

Putnam and Myers suggest four different approaches of Software Sizing Problem:

1. "Fuzzy Logic" Sizing:

- To apply this approach, the planner must identify the type of application.
- Although personal experience can be used, the planner should also have access to historical database of projects so that estimates can be compared to actual experience.

2. Function Point Sizing:

- The planner develops estimates of the information domain characteristics.

3. Standard Component Sizing:

- Standard components for an information system are subsystems, modules, screens, reports, interactive programs, batch programs, files, LOC and object-level instructions.
- Uses historical project data to determine the delivered size per standard component.

4. Change Sizing:

- This approach is used when a project encompasses the use of existing software that must be modified in some way as part of a project.
- The planner estimates reuse, adding code, changing code, deleting code of modifications that must be accomplished.
- Using an "effort ratio" for each type of change, the size of the change may be estim

2. Problem based Estimation

- Lines of code and function points were described as measures from which productivity metrics can be computed. **(LOC/pm & FP/pm)**

- **LOC and FP data are used in two ways during software project estimation:**

1. As an estimation variable to "size" each element of the software .
2. As baseline metrics collected from past projects and used in conjunction with estimation variables to develop cost and effort projections.

➢A three-point or expected value can then be computed

$S = (s_{opt} + 4s_m + s_{pess})/6$

- S = expected-value for the estimation variable (size)
- s_{opt} = optimistic value
- s_m = most likely value
- s_{pess} = pessimistic value

Examples of LOC & FP

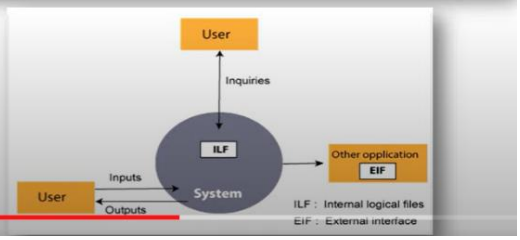
1. LOC (Lines of Code):

Project	LOC	Cost SK	Efforts (Persons/ Month)	Documentation	Errors
A	10000	110	18	365	39
B	12000	115	20	370	45
C	15400	130	25	400	32

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
Estimated lines of code	*33,200*

2. FP(Functional Points)

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
Count total						*320*



3. Process based Estimation

- Most common technique for estimating project is to estimate on the process that will be used.
- The process is decomposed into a relatively small set of tasks and the effort required to accomplish each task is estimated.
- Once problem functions & process activities are decided, the planner estimates the effort (e.g., person-months) that will be required to accomplish each software process activity for each software function.
- Average labor rates (cost/unit effort) are then applied to the effort estimated for each process activity.
- It is very likely the labor rate will vary for each task.
- Senior staff heavily involved in early activities are generally more expensive than junior

COCOMO Model:

The Constructive Cost Model (COCOMO) is a software cost estimation model that helps predict the effort, cost, and schedule required for a software development project. Developed by Barry Boehm in 1981, COCOMO uses a mathematical formula based on the size of the software project, typically measured in lines of code (LOC).

The COCOMO Model is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality. It was proposed by Barry Boehm in 1981 and is based on the study of 63 projects, which makes it one of the best-documented models.

The key parameters that define the quality of any software product, which are also an outcome of COCOMO, are primarily effort and schedule:

1. **Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.
2. **Schedule:** This simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, and months.

Types of Projects in the COCOMO Model

In the COCOMO model, software projects are categorized into three types based on their complexity, size, and the development environment. These types are:

1. **Organic:** A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.
2. **Semi-detached**: A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environments lie in between organic and embedded. The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience better guidance and creativity. Eg: Compilers or different Embedded Systems can be considered Semi-Detached types.
3. **Embedded:** A software project requiring the highest level of complexity, creativity, and experience requirement falls under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

Comparison of these three types of Projects in COCOMO Model

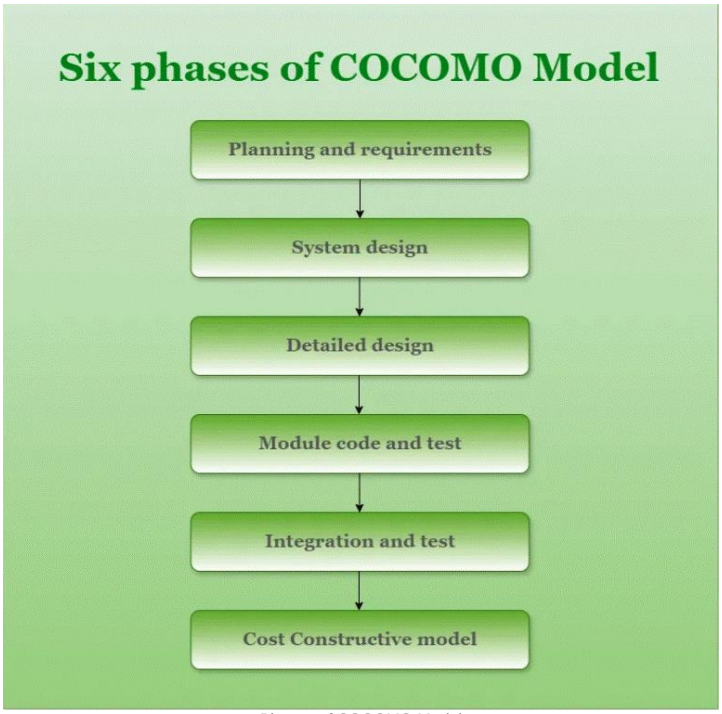
Aspects	Organic	Semidetached	Embedded
Project Size	2 to 50 KLOC	50-300 KLOC	300 and above KLOC
Complexity	Low	Medium	High
Team Experience	Highly experienced	Some experienced as well as inexperienced staff	Mixed experience, includes experts
Environment	Flexible, fewer constraints	Somewhat flexible, moderate constraints	Highly rigorous, strict requirements
Effort Equation	E = 2.4(400)1.05	E = 3.0(400)1.12	E = 3.6(400)1.20
Example	Simple payroll system	New system interfacing with existing systems	Flight control software

Detailed Structure of COCOMO Model

Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the software engineering process. The detailed model uses different effort multipliers for each cost driver attribute. In detailed COCOMO, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. **Planning and requirements**
2. **System design**
3. **Detailed design**
4. **Module code and test**
5. **Integration and test**
6. **Cost Constructive model**



Phases of COCOMO Model

Different models of COCOMO have been proposed to predict the cost estimation at different levels, based on the amount of accuracy and correctness required. All of these models can be applied to a variety of projects, whose characteristics determine the value of the constant to be used in subsequent calculations. These characteristics of different system types are mentioned below. Boehm's definition of organic, semidetached, and embedded systems:

Importance of the COCOMO Model

1. **Cost Estimation:** To help with resource planning and project budgeting, COCOMO offers a methodical approach to software development cost estimation.
2. **Resource Management:** By taking team experience, project size, and complexity into account, the model helps with efficient resource allocation.
3. **Project Planning**: COCOMO assists in developing practical project plans that include attainable objectives, due dates, and benchmarks.
4. **Risk management**: Early in the development process, COCOMO assists in identifying and mitigating potential hazards by including risk elements.
5. **Support for Decisions**: During project planning, the model provides a quantitative foundation for choices about scope, priorities, and resource allocation.
6. **Benchmarking**: To compare and assess various software development projects to industry standards, COCOMO offers a benchmark.
7. **Resource Optimization:** The model helps to maximize the use of resources, which raises productivity and lowers costs.

Types of COCOMO Model

There are three types of COCOMO Model:

- Basic COCOMO Model
- Intermediate COCOMO Model
- Detailed COCOMO Model

1. Basic COCOMO Model

The Basic COCOMO model is a straightforward way to estimate the effort needed for a software development project. It uses a simple mathematical formula to predict how many person-months of work are required based on the size of the project, measured in thousands of lines of code (KLOC).

It estimates effort and time required for development using the following expression:

$E = a*(KLOC)_b\ PM$

$Tdev = c*(E)_d$

Person required = Effort/ Time

Where,

E is effort applied in Person-Months

KLOC is the estimated size of the software product indicate in Kilo Lines of Code

Tdev is the development time in months

a, b, c are constants determined by the category of software project given in below table.

The above formula is used for the cost estimation of the basic COCOMO model and also is used in the subsequent models. The constant values a, b, c, and d for the Basic Model for the different categories of the software projects are:

Software Projects	a	b	C	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

1. The effort is measured in Person-Months and as evident from the formula is dependent on Kilo-Lines of code. The development time is measured in months.
2. These formulas are used as such in the Basic Model calculations, as not much consideration of different factors such as reliability, and expertise is taken into account, henceforth the estimate is rough.

Example of Basic COCOMO Model

Suppose that a Basic project was estimated to be 400 KLOC (kilo lines of code). Calculate effort and time for each of the three modes of development. All the constants value provided in the following table:

Solution

From the above table we take the value of constant a,b,c and d.

1. For organic mode,
   - effort = $2.4 \times (400)^{1.05} \approx 1295$ person-month.
   - dev. time = $2.5 \times (1295)^{0.38} \approx 38$ months.
2. For semi-detach mode,
   - effort = $3 \times (400)^{1.12} \approx 2462$ person-month.
   - dev. time = $2.5 \times (2462)^{0.35} \approx 38$ months.
3. For Embedded mode,
   - effort = $3.6 \times (400)^{1.20} \approx 4772$ person-month.
   - dev. time = $2.5 \times (4772)^{0.32} \approx 38$ months.

2. Intermediate COCOMO Model

The basic COCOMO model assumes that the effort is only a function of the number of lines of code and some constants evaluated according to the different software systems. However, in reality, no system's effort and schedule can be solely calculated based on Lines of Code. For that, various other factors such as reliability, experience, and Capability. These factors are known as **Cost Drivers (multipliers)** and the Intermediate Model utilizes 15 such drivers for cost estimation.

Classification of Cost Drivers and their Attributes:

The cost drivers are divided into four categories

Product attributes:

- Required software reliability extent
- Size of the application database
- The complexity of the product

Hardware attributes

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

Personal attributes

- Analyst capability
- Software engineering capability
- Application experience
- Virtual machine experience
- Programming language experience

Project attributes

- Use of software tools
- Application of software engineering methods

- Required development schedule

Each of the 15 such attributes can be rated on a six-point scale ranging from “very low” to “extra high” in their relative order of importance. Each attribute has an effort multiplier fixed as per the rating. Table give below represents Cost Drivers and their respective rating:
*The **Effort Adjustment Factor (EAF)** is determined by multiplying the effort multipliers associated with each of the 15 attributes.*

The Effort Adjustment Factor (EAF) is employed to enhance the estimates generated by the basic COCOMO model in the following expression:
Intermediate COCOMO Model equation:
$E = a*(KLOC)_b * EAF\ PM$
$Tdev = c*(E)_d$
Where,

- *E is effort applied in Person-Months*
- *KLOC is the estimated size of the software product indicate in Kilo Lines of Code*
- *EAF is the Effort Adjustment Factor (EAF) is a multiplier used to refine the effort estimate obtained from the basic COCOMO model.*
- *Tdev is the development time in months*
- *a, b, c are constants determined by the category of software project given in below table.*

The constant values a, b, c, and d for the Basic Model for the different categories of the software projects are:

Software Projects	a	b	C	d
Organic	3.2	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

3. Detailed COCOMO Model
Detailed COCOMO goes beyond Basic and Intermediate COCOMO by diving deeper into project-specific factors. It considers a wider range of parameters, like team experience, development practices, and software complexity. By analyzing these factors in more detail, Detailed COCOMO provides a highly accurate estimation of effort, time, and cost for software projects. It’s like zooming in on a project’s unique characteristics to get a clearer picture of what it will take to complete it successfully.

CASE Studies and Examples

1. **NASA Space Shuttle Software Development:** NASA estimated the time and money needed to build the software for the Space Shuttle program using the COCOMO model. NASA was able to make well-informed decisions on resource allocation and project scheduling by taking into account variables including project size, complexity, and team experience.
2. **Big Business Software Development:** The COCOMO model has been widely used by big businesses to project the time and money needed to construct intricate business software systems. These organizations were able to better plan and allocate resources for their software projects by using COCOMO’s estimation methodology.
3. **Commercial Software goods:** The COCOMO methodology has proven advantageous for software firms that create commercial goods as well. These businesses were able to decide on pricing, time-to-market, and resource allocation by precisely calculating the time and expense of building new software products or features.
4. **Academic Research Initiatives:** To estimate the time and expense required to create software prototypes or carry out experimental studies, academic research initiatives have employed COCOMO. Researchers were able to better plan their projects and allocate resources by using COCOMO’s estimate approaches.

Advantages of the COCOMO Model

1. **Systematic cost estimation:** Provides a systematic way to estimate the cost and effort of a software project.
2. **Helps to estimate cost and effort:** This can be used to estimate the cost and effort of a software project at different stages of the development process.
3. **Helps in high-impact factors:** Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
4. **Helps to evaluate the feasibility of a project:** This can be used to evaluate the feasibility of a software project by estimating the cost and effort required to complete it.

Disadvantages of the COCOMO Model

1. **Assumes project size as the main factor:** Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.
2. **Does not count development team-specific characteristics:** Does not take into account the specific characteristics of the development team, which can have a significant impact on the cost and effort of a software project.
3. **Not enough precise cost and effort estimate:** This does not provide a precise estimate of the cost and effort of a software project, as it is based on assumptions and averages.

Best Practices for Using COCOMO

1. **Recognize the Assumptions Underpinning the Model:** Become acquainted with the COCOMO model's underlying assumptions, which include its emphasis on team experience, size, and complexity. Understand that although COCOMO offers useful approximations, project results cannot be predicted with accuracy.
2. **Customize the Model**: Adapt COCOMO's inputs and parameters to your project's unique requirements, including organizational capacity, development processes, and industry standards. By doing this, you can be confident that the estimations produced by COCOMO are more precise and appropriate for your situation.
3. **Utilize Historical Data:** To verify COCOMO inputs and improve estimating parameters, collect and examine historical data from previous projects. Because real-world data takes project-specific aspects and lessons learned into account, COCOMO projections become more accurate and reliable.
4. **Verify and validate:** Compare COCOMO estimates with actual project results, and make necessary adjustments to estimation procedures in light of feedback and lessons discovered. Review completed projects to find errors and enhance future project estimation accuracy.
5. **Combine with Other Techniques:** To reduce biases or inaccuracies in any one method and to triangulate results, add COCOMO estimates to other estimation techniques including expert judgment, similar estimation, and bottom-up estimation.